pfi

Python for Fixed Income

COURSE 5B · TIME SERIES MODELS

Lag Features and Gradient Boosting

WEEK 2 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: six steps

Six steps

1. Build lag features, shift first and roll second
2. Prove that no feature uses its own date
3. Show a leak on purpose, and its score
4. Score LightGBM walk-forward
5. Every setting, sweeps, choose on development
6. Importance within folds; the size of the change

The question

Does a model that takes many inputs get closer than the last value? Development years only: 2 Jan 2015 to 30 Dec 2022, 2,001 dates. The honest finding comes first.

More inputs means more places to leak.

Names for a model's output

A model's output is a fitted change, or the model's value for that date. The baseline:

Allowed names

fitted change in bp

the model's one-step value

the last value carried forward

Never called

a forecast or a prediction

a projection or an outlook

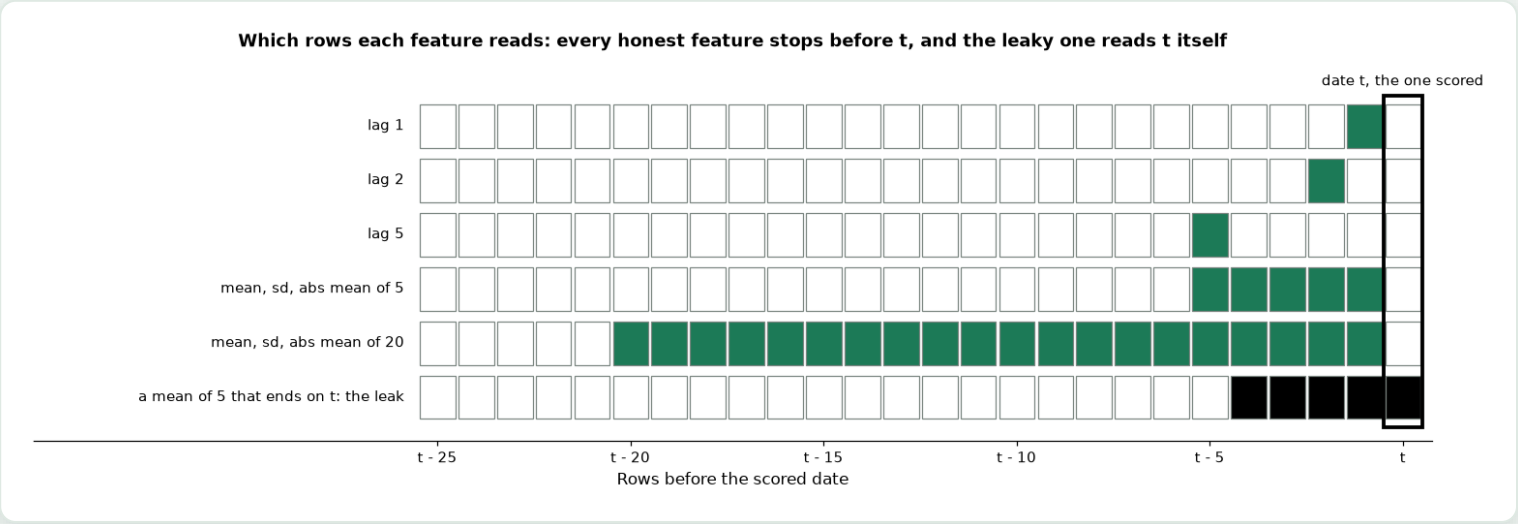
a signal, a call, a view on rates

Every score is on dates already in the file, from a model fitted on earlier dates.

How lag features work

The rule	A date may use rows before it and nothing on or after it
A lag	The value some rows back; a rolling statistic summarises the last w rows
The order	Shift first, then roll: every window ends on the row before the date
The trap	A window on the unshifted column ends on the date, with its own change inside
What comes out	One row per date, one column per feature, missing in the first rows

Which rows each feature reads



Feature	2015-02-03	2015-02-04
10 Yr lag 1	0.00	11.00
10 Yr mean 5	-3.00	-0.80
10 Yr sd 5	6.16	8.87
10 Yr abs mean 20	4.90	5.05
2 Yr lag 1	2.00	3.00
weekday	1.00	2.00

15 features: three lags, three statistics for each of two windows, the four other series' last changes, and two calendar fields. The first 20 rows have a missing feature.

Every honest feature stops one cell before the date t.

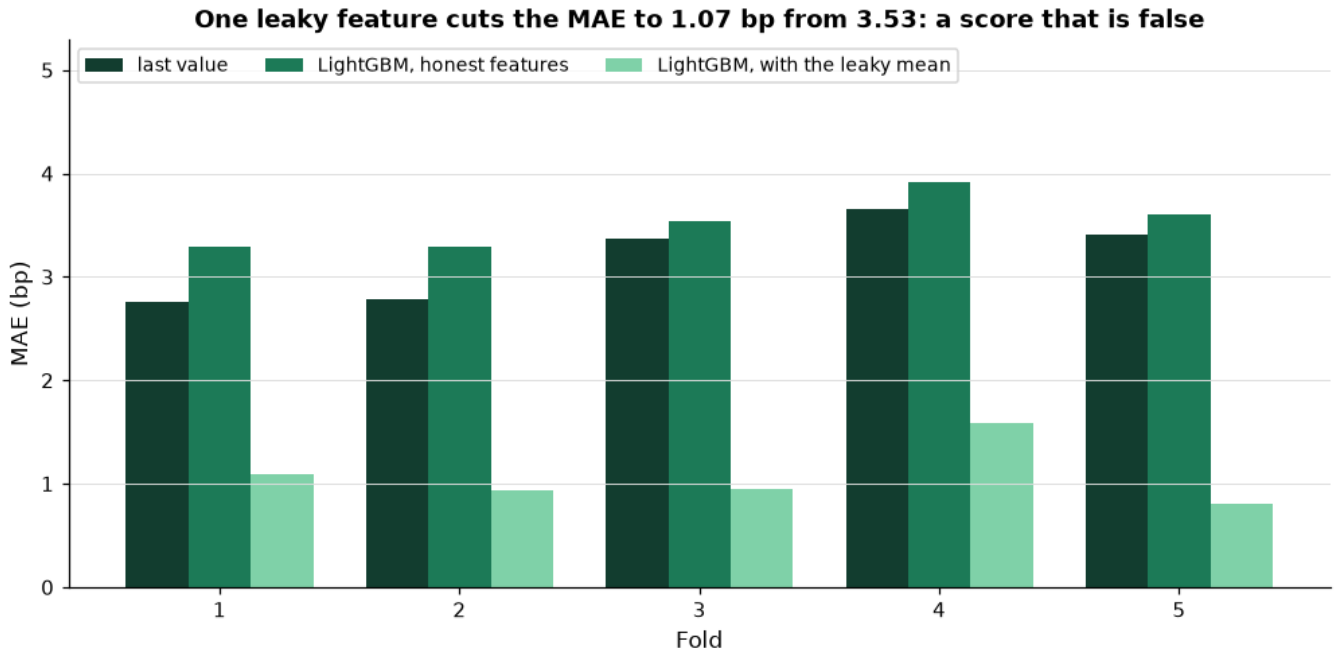
How LightGBM works

The model	One small tree at a time: each fits what the trees before it missed
The step	Each tree's correction is scaled down by the learning rate before it is added
The tree	Splits the rows by a question about one feature; each leaf gets a value
What you control	Trees, learning rate, leaves, fewest rows per leaf
Walk-forward	Fit once per fold on its training rows; a value for each test row from its features

How a leak is found

The problem	A leak is a column that happens to contain the answer: the score is better, and false
The test	Change every row from date t on, build the features again, compare those up to t
Honest	A feature at t reads rows before t, so changing rows from t on cannot move it
Leaky	A leaky feature moves
What comes out	Silence, or an error naming the feature and the date

What a leak does to a score



Mean MAE, bp

last value: 3.1937

LightGBM, honest: 3.5282

LightGBM, leaky: 1.0720

A mean of five that includes the scored change is a fifth of the answer in every row.

The proof: the feature 'leak' dated 28 May 2015 changed when the rows from 28 May 2015 on changed, so it uses a row that is not before its date

A model on a daily change that beats the last value by more than a few percent: look for the leak first.

Every setting of LGBMRegressor

Setting	Default	Tune first
objective	None	yes
learning_rate	0.1	yes
n_estimators	100	yes
num_leaves	31	yes
min_child_samples	20	yes
max_depth	-1	no
subsample	1.0	no
colsample_bytree	1.0	no
reg_lambda	0.0	no

How to read it

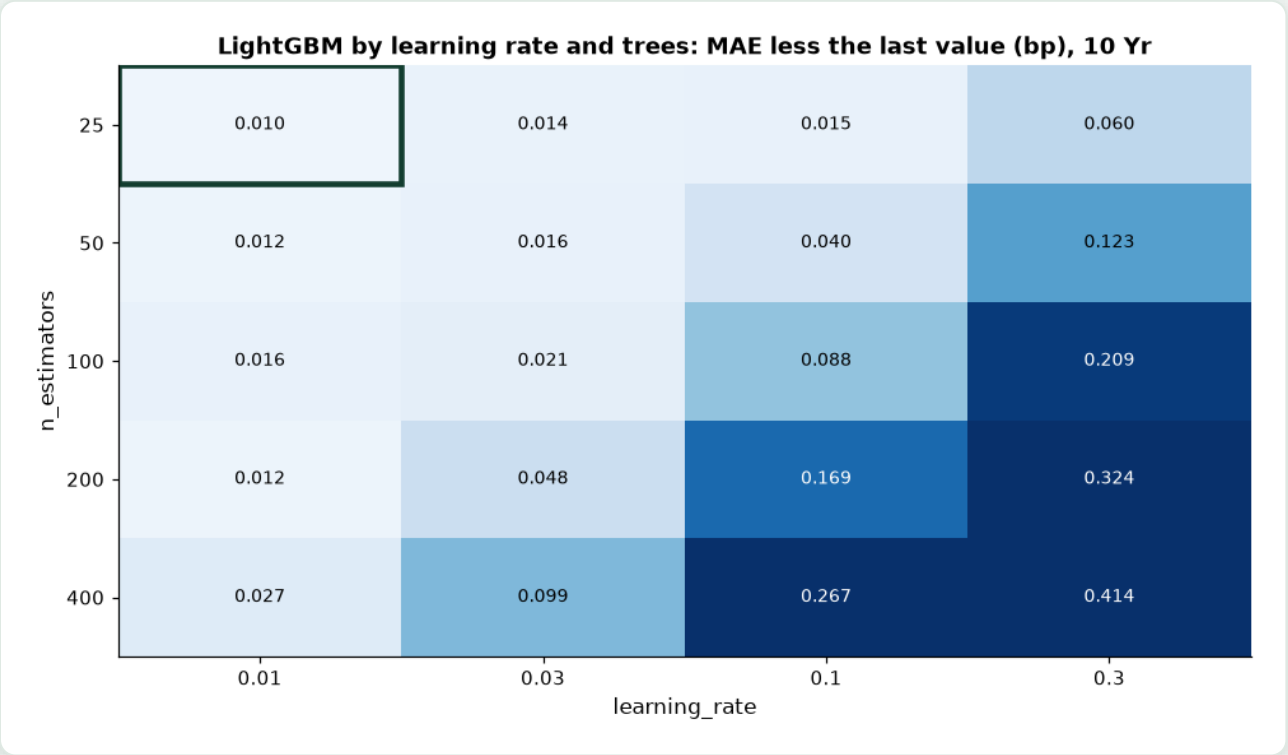
Five settings matter for a series like this: the objective, the learning rate, the number of trees, the leaves, and the fewest rows per leaf. The rest are guards, randomness, and housekeeping. The table has every setting the library has, and an assert stops the build if a new version adds one.

The defaults are read from the installed LightGBM's constructor, never typed.

The loss, and the learning rate against the trees

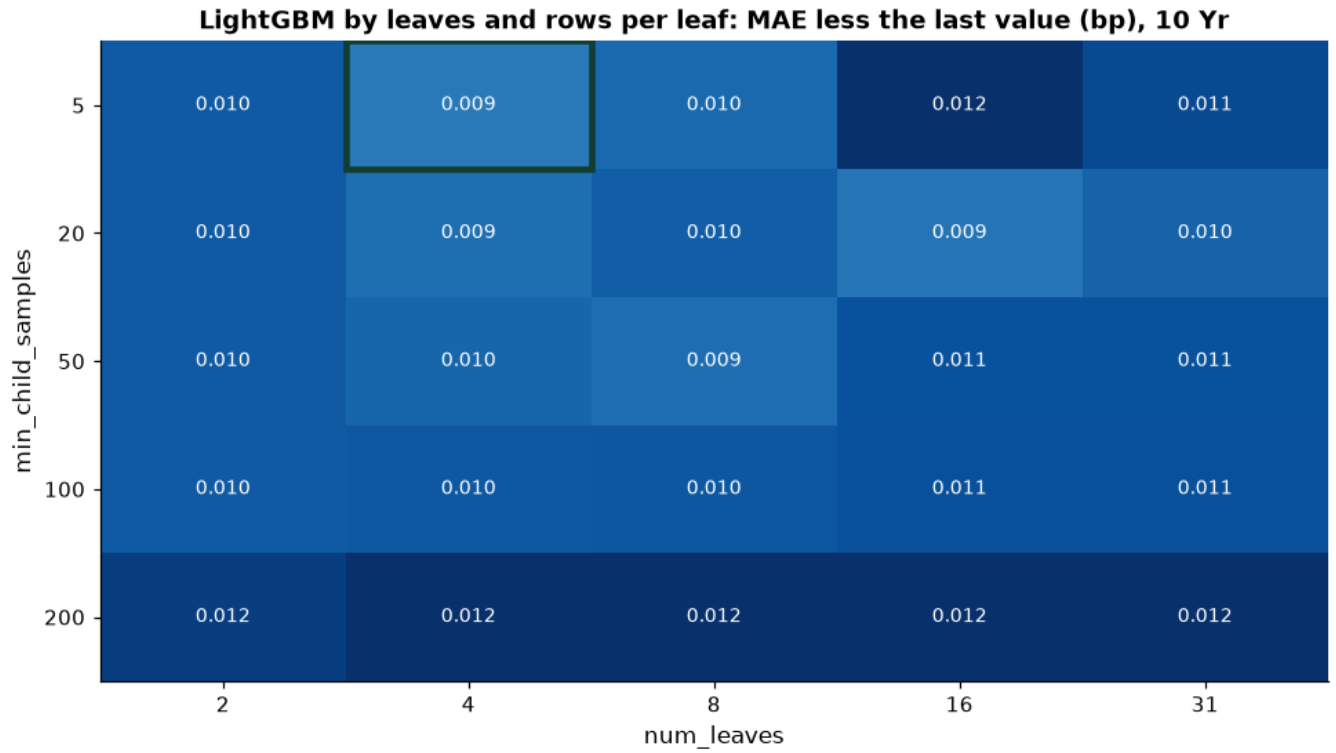
Objective	Mean MAE	Gap (bp)	Below
l2 (the default)	3.5282	0.3345	0
l1	3.3508	0.1572	0
huber	3.2812	0.0876	0

Squared error lets the few huge changes dominate the trees. Huber's loss squares small misses and counts large ones by size.



Lowest 0.0098 bp at learning rate 0.01, 25 trees: the corner where the model does the least. Configurations below the last value: 0 of 20.

Leaves and rows per leaf



How to read it

The lowest cell: 4 leaves, 5 rows, 0.0088 bp. The whole grid spans a few thousandths of a bp: at this learning rate and number of trees, the trees hardly matter. Chosen: {'objective': 'huber', 'learning_rate': 0.01, 'n_estimators': 25, 'num_leaves': 4, 'min_child_samples': 5}.

Chosen on the same development folds that score them: that favours it, and it still does not beat the last value.

Five models against the last value

10 Yr	Gap (bp)	MASE	Below
drift	0.0076	1.0024	0
AR(1)	0.0089	1.0028	1
smoothing	0.0059	1.0019	3
LGBM	0.3345	1.1047	0
LGBM chosen	0.0088	1.0027	0

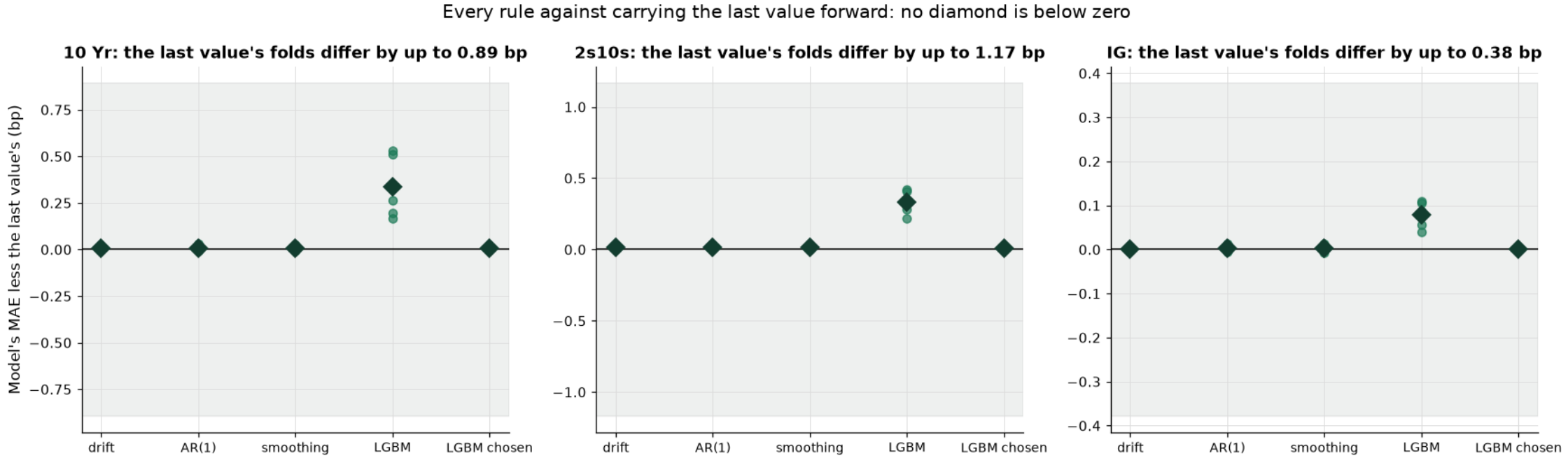
2s10s	Gap (bp)	MASE	Below
drift	0.0134	1.0062	0
AR(1)	0.0137	1.0064	0
smoothing	0.0135	1.0063	0
LGBM	0.3330	1.1545	0
LGBM chosen	0.0120	1.0056	0

IG	Gap (bp)	MASE	Below
drift	0.0003	1.0003	0
AR(1)	0.0024	1.0030	2
smoothing	0.0023	1.0028	2
LGBM	0.0796	1.0993	0
LGBM chosen	0.0003	1.0003	2

Gap: a model's MAE less the last value's, five-fold average. Below: folds, of 5, below the last value. The last value's own folds differ by 0.8929 bp (10 Yr), 1.1667 (2s10s), and 0.3778 (IG).

Every MASE is above 1. The bigger model is the worse model, and the chosen model is the smaller one.

Every gap against its band



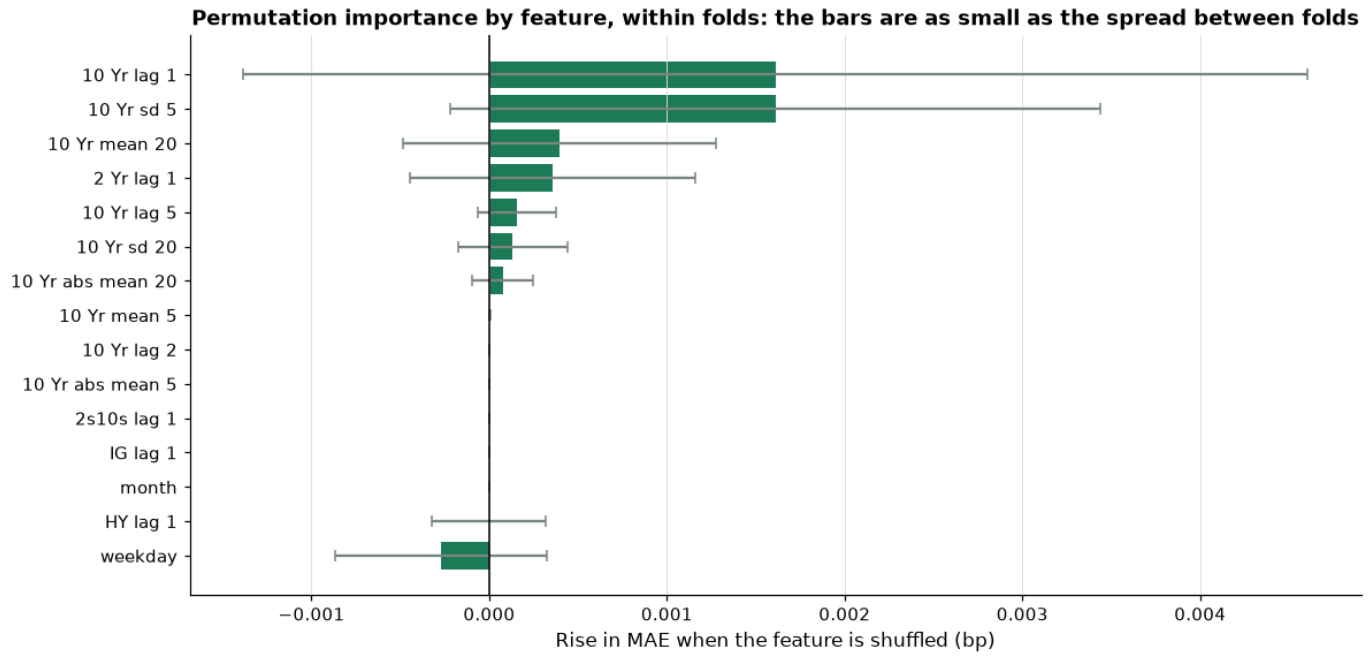
Four of five diamonds sit on zero, a hair above. LightGBM with the defaults sits well above it, with every fold above.

Dots are folds; diamonds are five-fold averages; the grey band is how far the last value's own folds differ.

How permutation importance works

The question	Does the fitted model need this feature to score well?
The method	Shuffle one feature among the test rows: its link to the target breaks
The read	How far the MAE rises; five shuffles per feature, averaged
Within folds	Each fold's own fitted model and test rows
What comes out	One number per feature per fold, in bp

What the importances say



How to read it

The largest mean rise is 10 Yr lag 1: 0.0016 bp. Read the axis: the chosen model uses almost nothing, so shuffling almost anything leaves its scores alone.

Run permutation importance after a model has beaten its baseline: on one that did not, it shows how little there is.

How the size model works

The memory	Day 7 found memory in the size of the change
The target	The absolute change; features from the absolute changes
The rules	Shift first, prove it, score walk-forward against a constant size
The comparison	Day 7's constant size, 20-row average, and GARCH on the same folds
What comes out	A model of how big the day's change is

The size of the change, five rules

Rule	Mean MAE	Ratio	Below
constant size	2.1302	1.0000	0
20-row average	2.0402	0.9578	3
GARCH(1,1)	2.0671	0.9704	3
LightGBM, defaults	2.2854	1.0729	1
LightGBM, chosen	2.1044	0.9879	5



The defaults fail again; the chosen settings find a little. A 20-row average and day 7's GARCH are closer.

When a simple rule does the job, that is the finding.

What the models show, and where they fail

What the models show

LightGBM on lag features was no closer than the last value on the five-fold average: MASE from 1.0003 to 1.1545 across the three series and five models. On the size of the change, the chosen settings were a little closer than a constant.

Where they fail

Level: more inputs made the score worse

Defaults: a model for data with structure in it

Leak: one feature cut the MAE to a third

Importances: they rank noise

A walk-forward error on past dates is not evidence about any later date, and not a view on rates.

Excel side by side

Excel side by side (tested in Excel)

`=B21`	lag 1
`=AVERAGE(B17:B21)`	mean, 5 rows before
`=STDEV.S(B17:B21)`	sd, 5 rows before
`=SUMPRODUCT(ABS(B2:B21))/20`	mean abs, 20 rows before
`=AVERAGE(B18:B22)`	the leak: ends on the row
`=WEEKDAY(A22,2)-1`	weekday, Monday as 0

Largest gap from Python on any feature: 9e-14. Excel builds the features; it does not fit the model.

Save the day: a stash

1. Commit

The code, then the 10 Yr's scores

2. Experiment

The file is changed: status shows M

3. Stash

git stash push puts the change aside

4. Pop

git stash pop -q brings it back

```
Day 8: LightGBM scores
Day 8: tskit with lag_features, assert_features_past_only, and walk_forward_lgbm
```

The stash is listed under its name: wider grid, unfinished.

A stash is not a commit: it is a place for work that is not ready to be one.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which dates fit, which dates score, what may be filled: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the rolling window ends on the date: feature 'mean' dated 20 May 2020 changed when the rows from 20 May 2020 on changed

Code that runs is not the same as code that is right.

What you can do now

- Lag features: shift first, then roll
- assert_features_past_only: nothing uses its own date
- A leak scores far too well, and only a proof catches it
- Every LGBMRegressor setting, and sweeps: no model beat the last value
- On the size of the change: a little closer than a constant

Week 2, Day 4 Two series: cointegration and mean reversion